Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Thread (İş Parçacığı) Kavramı
  - Bir process içinde birden fazla bağımsız akış (thread) oluşturma mekanizmasıdır. Thread, process'in temel bileşenlerinden (kod, data, register, stack) paylaşımlı olup sadece register ve stack ayrıdır.
  - Thread'in temel motivasyonları:
    - CPU utilization artırmak.
    - Uygulamanın farklı parçalarını paralel yürütmek.
    - İnteraktif yapıyı geliştirmek (UI thread + worker thread).
    - Kodu basitleştirmek ve modüler hale getirmek.
- Thread vs Process
  - Process: Kendi adres alanı vardır. Oluşturma maliyeti yüksek (kod, data, register, stack hepsi kopyalanır).
  - Thread: Aynı process'in adres alanını paylaşır. Sadece register ve stack ayrıdır. Oluşturma maliyeti düşük.
  - Bir process birden fazla thread barındırabilir; tüm thread'ler aynı kod, data ve dosya alanını paylaşır.
- Thread Modelleri (User vs Kernel Threads)
  - User threads: Uygulama geliştiricinin thread kütüphaneleri (POSIX pthreads, Windows threads, Java threads) ile oluşturduğu thread'ler.
  - Kernel threads: İşletim sistemi tarafından doğrudan desteklenen ve yönetilen thread'ler. Modern OS'ler (Linux, Windows, Solaris, macOS) varsayılan olarak kernel thread kullanır.
  - Üç eşleme modeli:
    - **Many-to-One (N:1)**: Birçok user thread tek bir kernel thread'e eşlenir. Bir user thread blocking call yaparsa tüm kernel thread (ve bağlı tüm user thread'ler) bloke olur. Ekonomik ama esnek değil.
    - **One-to-One (1:1)**: Her user thread için bir kernel thread oluşturulur. Blocking call'lar problem olmaz; ancak kaynak kullanımı fazladır.
    - **Many-to-Many (N:M)**: Birçok user thread, daha az (eşit veya daha fazla olmayan) sayıda kernel thread'e eşlenir. Her iki modelin avantajlarını birleştirir.
    - **Two-level**: Kritik thread'ler 1:1, diğerleri N:M şeklinde.
- Thread Kütüphaneleri
  - POSIX pthreads: Unix/Linux için thread standardı. Fonksiyonlar: `pthread_create`, `pthread_exit`, `pthread_join`, `pthread_detach`, `pthread_self`, `pthread_equal`.
  - Windows threads: Win32 API ile.
  - Java threads: `Thread` sınıfı veya `Runnable` arayüzü ile; JVM tarafından yönetilir.
- Thread Avantajları
  - Ekonomi: Process oluşturmaktan daha ucuz; bellek ve kaynak paylaşımı.
  - Modülerlik: Karmaşık uygulamaları parçalara ayırma.
  - Hız: Context switch process'ler arası context switch'ten daha hızlı.
  - İletişim: Aynı process'teki thread'ler doğrudan paylaşımlı bellek üzerinden haberleşir (IPC maliyeti yok).
- Thread Dezavantajları
  - Senkronizasyon karmaşıklığı: Paylaşımlı kaynaklara erişim senkronize edilmelidir (race condition).

  - Test ve debug zorluğu: Multi-threaded uygulamaları test etmek ve debug etmek tek thread'li uygulamalardan çok daha zordur.
  - Kernel desteği gereksinimi: Modern OS'ler kernel seviyesinde thread desteği sağlar.
- Multi-core Sistemler ve Threading
  - Multi-core işlemciler, gerçek anlamda paralel çalışma sağlar. Ancak bir uygulamanın multi-core'dan yararlanabilmesi için multi-threaded olması gerekir.
  - Intel çekirdek başına 2 thread (Hyper-Threading); RISC tabanlı işlemcilerde 4-8 thread/core olabilir.
  - ALU (Aritmetik-Logic Unit) birden fazla olursa, aynı anda farklı aritmetik işlemler yapılabilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Thread Kütüphanesi Implementasyonu
  - Hoca iki yöntem açıklar: (1) Kütüphane tamamen user seviyesinde (user-mode thread library), (2) Kütüphane kernel seviyesinde (kernel-mode thread library). POSIX standart olarak thread davranış modelini tanımlar; implementasyonu sisteme bırakır.
- 8 Çekirdek 16 Thread Meselesi
  - Hoca bir soruya cevap verir: "8 çekirdek 16 thread" ifadesindeki thread sayısı, işlemcinin kaç tane fiziksel register seti barındırdığına bağlıdır. Intel her çekirdek için 2 register seti (2 thread) sağlar; RISC tabanlı işlemcilerde 4-8 thread/core olabilir. ALU birden fazlaysa aynı anda farklı aritmetik işlemler yapılabilir.
- Multi-threaded Server Mimarisi
  - Hoca, web sunucu örneğini verir: Apache varsayılan olarak 5 process × 30 thread = 150 thread ile başlar. Multi-process ana yapıyı, multi-thread istek işlemeyi sağlar. Bir thread'te hata olursa sadece o thread etkilenir, tüm process çökmez.
- Debug Zorluğu
  - Hoca özellikle vurgular: "printf bile debug için yazdırılsa kimin tarafından yazdırıldığı sorusu ortaya çıkıyor." Multi-threaded uygulamada printf, kernel'ı bloke edebilir, threadler arası sırayı değiştirebilir. Bu yüzden test ve debug oldukça sıkıntılı olabilir.
- Thread'lerin Çalışma Hakkında Karar
  - Hoca vurgular: Thread'ler CPU-bound (çok hesaplama) ve I/O-bound (çok I/O) olabilir. Bir thread I/O bloğuna girerse diğer thread'ler çalışabilir. Bu yüzden tek core'da bile threading anlamlı olabilir.

Kısa Tekrar Notları

- Thread = process içindeki bağımsız akış.
- Thread paylaşır: kod, data, dosya. Ayrı tutar: register, stack.
- Üç thread modeli: Many-to-One, One-to-One, Many-to-Many.
- POSIX pthreads temel thread standardı.
- Hyper-Threading: çekirdek başına 2 thread (Intel).
- Multi-core için multi-threaded uygulama gerekli.
- Multi-threaded debug zorluğu önemli bir dezavantajdır.

Detaylı Açıklamalar

Ders 7, thread (iş parçacığı) kavramını derinlemesine ele alır. Thread, modern işletim sistemlerinin temel yapı taşlarından biridir. Bir process içinde birden fazla bağımsız akış (thread) oluşturma imkânı sağlar.

Thread'in temel motivasyonu CPU utilization'ı artırmaktır. Process'ler arası context switch maliyetli olduğundan, bir process içinde birden fazla thread oluşturmak daha ekonomiktir. Bunun dışında: uygulamanın farklı parçalarını paralel yürütmek, interaktif yapıyı geliştirmek (UI thread + arka plan worker thread), kodu modüler hale getirmek.

Thread vs Process farkı detaylı şekilde açıklanır. Process = kendi adres alanı olan bağımsız çalışma birimi. Oluşturulması maliyetli (kod, data, register, stack kopyalanır). Thread = pro-

cess’in adres alanını paylaşan bağımsız akış. Sadece register ve stack ayrıdır. Oluşturulması ucuz, context switch hızlı.

Thread modelleri (user vs kernel threads) detaylı şekilde anlatılır. Üç eşleme modeli vardır: - Many-to-One (N:1): Çok sayıda user thread tek bir kernel thread’e eşlenir. Thread library user space’tedir. Avantaj: ekonomik (kernel kaynağı az kullanılır). Dezavantaj: Bir user thread blocking call yaptığında tüm kernel thread bloke olur, bağlı tüm user thread’ler etkilenir. - One-to-One (1:1): Her user thread için ayrı bir kernel thread oluşturulur. Blocking call problemi yoktur. Avantaj: paralellik yüksek. Dezavantaj: her thread için kernel kaynağı gerekir, oluşturma maliyeti yüksek. - Many-to-Many (N:M): Çok sayıda user thread, daha az sayıda kernel thread’e eşlenir. Her iki modelin avantajlarını birleştirir. Bir thread blocking call yaparsa diğerleri devam edebilir. - Two-level: Kritik thread’ler için 1:1, normal thread’ler için N:M.

Hoca, thread kütüphanelerinin implementasyonunu açıklar: (1) Tamamen user space’te (kullanıcı thread’leri yönetir, kernel haberdar değildir), (2) Tamamen kernel space’te (kernel thread’leri oluşturur ve yönetir), (3) Hibrit yaklaşım.

Thread avantajları detaylı sıralanır: Ekonomi (process oluşturmaktan ucuz, kaynak paylaşımı), Modülerlik (karmaşık uygulamaları parçalara ayırma), Hız (context switch process’ler arası context switch’ten hızlı), İletişim (aynı process’teki thread’ler doğrudan paylaşımlı bellek üzerinden haberleşir).

Thread dezavantajları da önemlidir: Senkronizasyon karmaşıklığı (paylaşımlı kaynaklara erişim senkronize edilmelidir), Test ve debug zorluğu (multi-threaded uygulamalar tek thread’li uygulamalardan çok daha zor test edilir, çünkü thread’lerin çalışma sırası belirsizdir).

Multi-core sistemler ve threading konusu da vurgulanır. Modern işlemcilerde Hyper-Threading teknolojisi ile her fiziksel çekirdek 2 mantıksal thread (iki register seti) barındırır. Bu sayede bir thread I/O veya bellek erişimi için beklerken, diğer thread hesaplama yapabilir. ALU birden fazlaysa (4-8), aynı anda farklı aritmetik işlemler yapılabilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.